

2024 PERSONAL FINANCE AND INVESTMENT ANALYSIS

(MYSQL EDITION)

MARK ANTHONY A. BUNA

Data Analyst

PROBLEM STATEMENT

- Managing personal finances often lacks structure, making it difficult to track where money goes, how investments perform, and whether financial goals are being met. This project uses MySQL to build a personal financial tracking system that organizes income, expenses, investments, savings, and debts — with the goal of uncovering trends and gaining clarity over financial health throughout 2024.

OBJECTIVE

- Create a MySQL database to track income, expenses, investments, savings, and debts, and analyze financial trends.
- Analyze financial activity to answer key questions for the year 2024:
 - What is the general financial overview for each month in 2024?*
 - How are expenses distributed across different categories, and what are the 3 major spending areas?*
 - How much have investments earned in terms of returns?*
 - What is the total debt and interest paid?*
 - What is the total net worth?*

FINAL PORTFOLIO DELIVERABLES

-  Built a MySQL-based financial tracking system with analysis queries, automated views, and full documentation hosted on GitHub.

LINK:

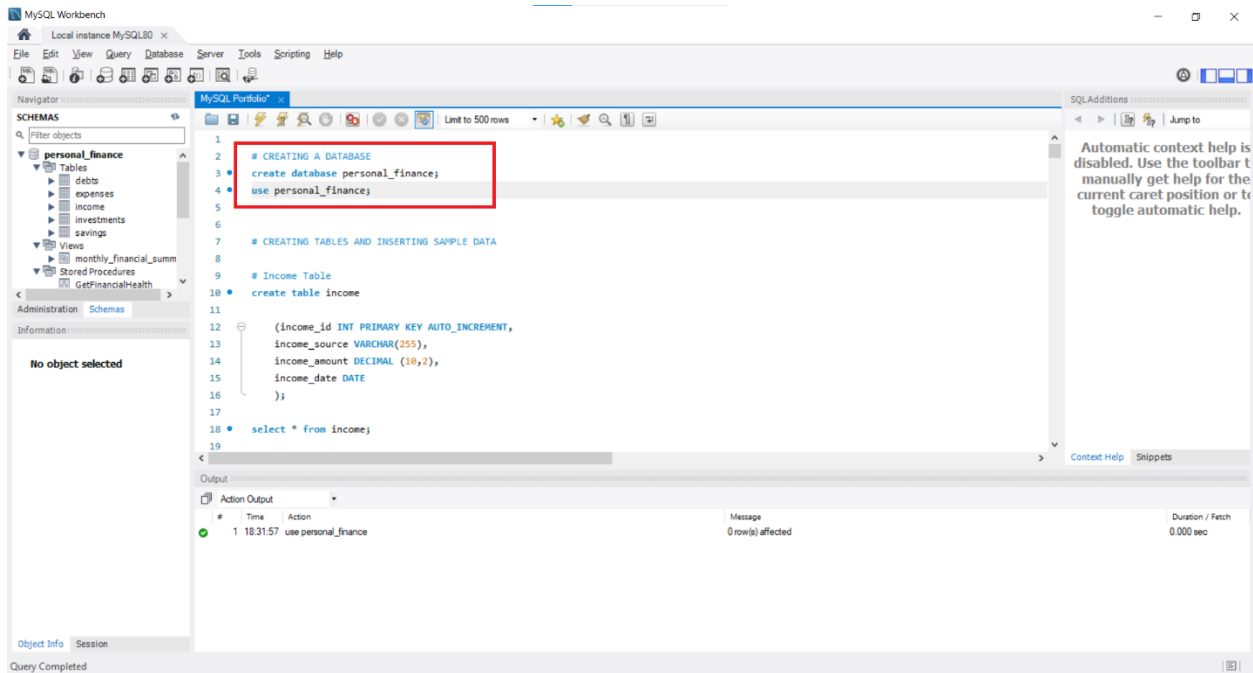
- Github: <https://github.com/marktheanalyst103/mysql-scripts/blob/main/2024%20Personal%20Finance%20and%20Investment%20Analysis.sql>



PROJECT STEPS

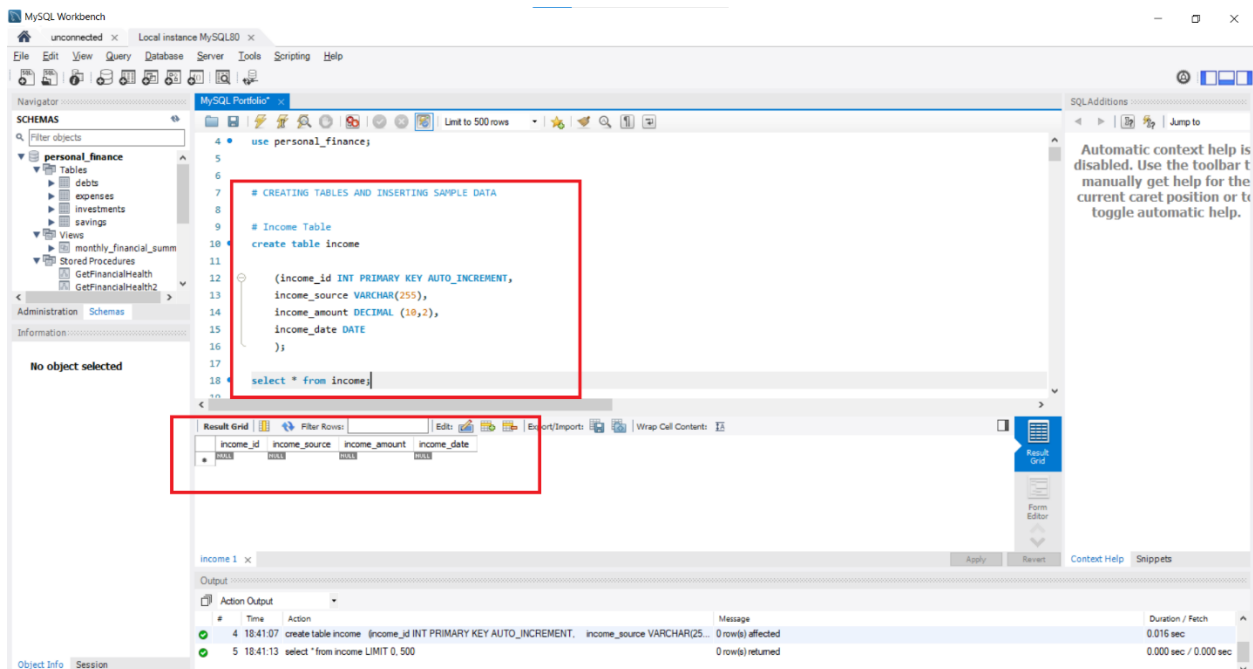
Step 1: Set Up MySQL Environment

- Install MySQL and set up the database.



Step 2: Create Tables & Insert Sample Data

- Define tables for *income*, *expenses*, *investments*, *savings*, and *debts*.
- Populate tables with sample financial data.



MySQL Workbench

Navigator

SCHEMAS

personal_finance

Tables

debts

expenses

investments

savings

Views

monthly_financial_summ

Stored Procedures

GetFinancialHealth

GetFinancialHealth2

Administration Schemas

Information

No object selected

MySQL Portfolio

Limit to 500 rows

15 income_date DATE

16);

17

18 select * from income;

19

20 Insert into income (income_source, income_amount, income_date)

21 values

22 ('Concentrix Company', 6802, '2024-03-01'),

23 ('Concentrix Company', 8311, '2024-03-15'),

24 ('Concentrix Company', 9044, '2024-03-27'),

25 ('Concentrix Company', 9916, '2024-04-12'),

26 ('Concentrix Company', 10760, '2024-04-26'),

27 ('Concentrix Company', 8096, '2024-05-10'),

28 ('Concentrix Company', 17995, '2024-05-24'),

29 ('Concentrix Company', 10520, '2024-06-07'),

30 ('Concentrix Company', 8151, '2024-06-21'),

31 ('Concentrix Company', 2500, '2024-06-25'),

32 ('Concentrix Company', 6376, '2024-07-05'),

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007

1008

1009

1010

1011

1012

1013

1014

1015

1016

1017

1018

1019

1020

1021

1022

1023

1024

1025

1026

1027

1028

1029

1030

1031

1032

1033

1034

1035

1036

1037

1038

1039

1040

1041

1042

1043

1044

1045

1046

1047

1048

1049

1050

1051

1052

1053

1054

1055

1056

1057

1058

1059

1060

1061

1062

1063

1064

1065

1066

1067

1068

1069

1070

1071

1072

1073

1074

1075

1076

1077

1078

1079

1080

1081

1082

1083

1084

1085

1086

1087

1088

1089

1090

1091

1092

1093

1094

1095

1096

1097

1098

1099

1100

1101

1102

1103

1104

1105

1106

1107

1108

1109

1110

1111

1112

1113

1114

1115

1116

1117

1118

1119

1120

1121

1122

1123

1124

1125

1126

1127

1128

1129

1130

1131

1132

1133

1134

1135

1136

1137

1138

1139

1140

1141

1142

1143

1144

1145

1146

1147

1148

1149

1150

1151

1152

1153

1154

1155

1156

1157

1158

1159

1160

1161

1162

1163

1164

1165

1166

1167

1168

1169

1170

1171

1172

1173

1174

1175

1176

1177

1178

1179

1180

1181

1182

1183

1184

1185

1186

1187

1188

1189

1190

1191

1192

1193

1194

1195

1196

1197

1198

1199

1200

1201

1202

1203

1204

1205

1206

1207

1208

1209

1210

1211

1212

1213

1214

1215

1216

1217

1218

1219

1220

1221

1222

1223

1224

1225

1226

1227

1228

1229

1230

1231

1232

1233

1234

1235

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

1253

1254

1255

1256

1257

1258

1259

1260

1261

1262

1263

1264

1265

1266

1267

1268

1269

1270

1271

1272

1273

1274

1275

1276

1277

1278

1279

1280

1281

1282

1283

1284

1285

1286

1287

1288

1289

1290

1291

1292

1293

1294

1295

1296

1297

1298

1299

1300

1301

1302

1303

1304

1305

1306

1307

1308

1309

1310

1311

1312

1313

1314

1315

1316

1317

1318

1319

1320

1321

1322

1323

1324

1325

1326

1327

1328

1329

1330

1331

1332

1333

1334

1335

1336

1337

1338

1339

1340

1341

1342

1343

1344

1345

1346

1347

1348

1349

1350

1351

1352

1353

1354

1355

1356

1357

1358

1359

1360

1361

1362

1363

1364

1365

1366

1367

1368

1369

1370

1371

1372

1373

1374

1375

1376

1377

1378

1379

1380

1381

1382

1383

1384

1385

1386

1387

1388

1389

1390

1391

1392

1393

1394

1395

1396

1397

1398

1399

1400

1401

1402

1403

1404

1405

1406

1407

1408

1409

1410

1411

1412

1413

1414

1415

1416

1417

1418

1419

1420

1421

1422

1423

1424

1425

1426

1427

1428

1429

1430

1431

1432

1433

1434

1435

1436

1437

1438

1439

1440

1441

1442

1443

1444

1445

1446

1447

1448

1449

1450

1451

1452

1453

1454

1455

1456

1457

1458

<

MySQL Workbench

unconnected x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

personal_finance

Tables

debts

expenses

investments

savings

Views

monthly_financial_summ

Stored Procedures

GetFinancialHealth

GetFinancialHealth2

Administration Schemas

Information

No object selected

MySQL Portfolio

Limit to 500 rows

```
2 # Expense Table
3 create table expenses
4 (expense_id INT PRIMARY KEY AUTO_INCREMENT,
5  expense_category VARCHAR(255),
6  expense_amount DECIMAL(10,2),
7  expense_date DATE
8 )
9
10
11 select * from expenses;
12
13 insert into expenses (expense_category, expense_amount, expense_date)
14 values
15 ('Other', 99, '2024-01-01'),
16 ('Physical', 60, '2024-01-06'),
17 ('Rent', 3300, '2024-01-06'),
18 ('Physical', 60, '2024-01-09'),
```

Result Grid

expense_id	expense_category	expense_amount	expense_date
1	Other	99.00	2024-01-01
2	Physical	60.00	2024-01-06
3	Rent	3300.00	2024-01-06
4	Physical	60.00	2024-01-09
5	Physical	60.00	2024-01-10
6	Physical	554.00	2024-01-11
7	Physical	490.00	2024-01-11
8	Physical	60.00	2024-01-12
9	Physical	155.00	2024-01-12
10	Physical	60.00	2024-01-13

expenses 8 x

Output

Session

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Data for the "expense" table

MySQL Workbench

unconnected x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

personal_finance

Tables

debts

expenses

investments

savings

Views

monthly_financial_summ

Stored Procedures

GetFinancialHealth

GetFinancialHealth2

Administration Schemas

Information

No object selected

MySQL Portfolio

Limit to 500 rows

```
861 # Investment Table
862 create table investments
863 (investment_id INT PRIMARY KEY AUTO_INCREMENT,
864  investment_category VARCHAR(255),
865  investment_amount DECIMAL(10,2),
866  investment_return DECIMAL(10,2),
867  investment_date DATE
868 )
869
870
871 select * from investments;
872
873 insert into investments (investment_category, investment_amount, investment_return, investment_date)
874 values
875 ('Pag-ibig Contribution', 105, 110.25, '2024-01-11'),
876 ('Pag-ibig Contribution', 1005, 1055.25, '2024-01-11'),
877 ('Banks', 20, 21, '2024-07-11'),
878 ('Banks', 25, 26.25, '2024-07-11'),
879 ('Banks', 10, 10.5, '2024-07-12'),
880 ('Lending', 500, 525, '2024-12-25'),
```

Result Grid

investment_id	investment_category	investment_amount	investment_return	investment_date
1	Pag-ibig Contribution	105.00	110.25	2024-01-11
2	Pag-ibig Contribution	1005.00	1055.25	2024-01-11
3	Banks	20.00	21.00	2024-07-11
4	Banks	25.00	26.25	2024-07-11
5	Banks	10.00	10.50	2024-07-12
6	Lending	500.00	525.00	2024-12-25
7	Lending	500.00	515.00	2024-12-25

investments 9 x

Output

Session

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Data for the "investment" table

MySQL Workbench

unconnected x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

personal_finance

Tables

debts

expenses

investments

savings

Views

monthly_financial_summ

Stored Procedures

GetFinancialHealth

GetFinancialHealth2

Administration Schemas

Information

No object selected

MySQL Portfolio

Limit to 500 rows

```
888 # Savings Table
889 create table savings
890 (
891     saving_id INT PRIMARY KEY AUTO_INCREMENT,
892     saving_category VARCHAR(255),
893     saving_amount DECIMAL (10,2),
894     saving_return DECIMAL (10,2),
895     saving_date DATE
896 )
897
898 select * from savings;
899
900 insert into savings (saving_category, saving_amount, saving_return, saving_date)
901 values
902 ('Emergency Fund', 28, 29.4, '2024-01-30'),
903 ('Emergency Fund', 20, 21, '2024-01-30'),
904 ('Emergency Fund', 15, 15.75, '2024-02-02'),
905 ('Emergency Fund', 17, 17.85, '2024-02-02'),
906 ('Emergency Fund', 13, 13.65, '2024-02-06'),
907 ('Emergency Fund', 13, 13.65, '2024-02-06');
```

Result Grid

	saving_id	saving_category	saving_amount	saving_return	saving_date
1	1	Emergency Fund	28.00	29.40	2024-01-30
2	2	Emergency Fund	20.00	21.00	2024-01-30
3	3	Emergency Fund	15.00	15.75	2024-02-02
4	4	Emergency Fund	17.00	17.85	2024-02-02
5	5	Emergency Fund	13.00	13.65	2024-02-06
6	6	Emergency Fund	13.00	13.65	2024-02-06
7	7	Emergency Fund	13.00	13.65	2024-02-06

savings 10 x

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Apply Revert Context Help Snippets

Data for the "savings" table

MySQL Workbench

unconnected x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

personal_finance

Tables

debts

expenses

investments

savings

Views

monthly_financial_summ

Stored Procedures

GetFinancialHealth

GetFinancialHealth2

Administration Schemas

Information

No object selected

MySQL Portfolio

Limit to 500 rows

```
986 # Debts Table
987 create table debts
988 (
989     debt_id INT PRIMARY KEY AUTO_INCREMENT,
990     debt_category VARCHAR(255),
991     debt_amount DECIMAL (10,2),
992     debt_return DECIMAL (10,2),
993     debt_date DATE
994 )
995
996 select * from debts;
997
998 insert into debts (debt_category, debt_amount, debt_return, debt_date)
999 values
1000 ('Credit Card Debt', 250, 262.5, '2024-01-14'),
1001 ('Credit Card Debt', 65, 68.25, '2024-03-13'),
1002 ('Credit Card Debt', 13, 13.65, '2024-03-27'),
1003 ('Debt from Family/Friends', 300, 315, '2024-01-12'),
1004 ('Debt from Family/Friends', 500, 525, '2024-01-21'),
1005 ('Debt from Family/Friends', 75, 78.75, '2024-03-15');
```

Result Grid

	debt_id	debt_category	debt_amount	debt_return	debt_date
1	1	Credit Card Debt	250.00	262.50	2024-01-14
2	2	Credit Card Debt	65.00	68.25	2024-03-13
3	3	Credit Card Debt	13.00	13.65	2024-03-27
4	4	Debt from Family/Friends	300.00	315.00	2024-01-12
5	5	Debt from Family/Friends	500.00	525.00	2024-01-21
6	6	Debt from Family/Friends	75.00	78.75	2024-03-15
7	7	Debt from Family/Friends	119.00	124.05	2024-03-27

debts 11 x

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Apply Revert Context Help Snippets

Data for the "debts" table

💰 Step 3: Financial Analysis Queries

3.1 Total Monthly Income vs. Expenses

- Query to compare total monthly income and expenses.

The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query editor contains two SQL queries. The first query, titled '# EXPENSES per Month', is highlighted with a green box and is the active query. The second query, titled '# Income per month', is highlighted with a red box. The result grid shows the output of the first query, with columns 'monthly_report' and 'total_expenses'. The data is grouped by month from 2024-01 to 2024-07, with a total for 2024-07.

```
## FINANCIAL ANALYSIS QUERIES

# TOTAL MONTHLY INCOME VS EXPENSES
# Expenses per Month
select
  date_format(expense_date, '%Y-%m') as monthly_report,
  sum(expense_amount) as total_expenses
from expenses
group by monthly_report
;
```

EXPENSES PER MONTH

monthly_report	total_expenses
2024-01	13258.00
2024-02	12138.00
2024-03	57316.00
2024-04	36724.00
2024-05	41487.68
2024-06	39995.08
2024-07	60110.74

The screenshot shows the MySQL Workbench interface with a query editor and a result grid. The query editor contains two SQL queries. The first query, titled '# EXPENSES per Month', is highlighted with a red box. The second query, titled '# Income per month', is highlighted with a green box and is the active query. The result grid shows the output of the second query, with columns 'monthly_report' and 'total_income'. The data is grouped by month from 2024-03 to 2024-08, with a total for 2024-17.

```
## FINANCIAL ANALYSIS QUERIES

# TOTAL MONTHLY INCOME VS EXPENSES
# Expenses per Month
select
  date_format(expense_date, '%Y-%m') as monthly_report,
  sum(expense_amount) as total_expenses
from expenses
group by monthly_report
;
```

INCOME PER MONTH

```
# Income per month
select
  date_format(income_date, '%Y-%m') as monthly_report,
  sum(income_amount) as total_income
from income
group by monthly_report
;
```

monthly_report	total_income
2024-03	24157.00
2024-04	20676.00
2024-05	26091.00
2024-06	21171.00
2024-07	37350.00
2024-08	14045.00
2024-17	6050.00

MySQL Workbench

Navigator

SCHEMAS

personal_finance

Tables

debts

expenses

income

investments

savings

Views

monthly_financial_summ

Stored Procedures

GetFinancialHealth

Administration Schemas

Information

No object selected

SQL Editor

```

1020 # Monthly Income and Expenses: Joining and Combining Data
1021 SELECT
1022     income.monthly_report,
1023     income.total_income,
1024     expenses.total_expenses
1025 FROM
1026     (SELECT
1027         DATE_FORMAT(income_date, '%Y-%m') AS monthly_report,
1028         SUM(income_amount) AS total_income
1029     FROM income
1030     GROUP BY monthly_report) AS income
1031 LEFT JOIN
1032     (SELECT
1033         DATE_FORMAT(expense_date, '%Y-%m') AS monthly_report,
1034         SUM(expense_amount) AS total_expenses
1035     FROM expenses
1036     GROUP BY monthly_report) AS expenses
1037 ON income.monthly_report = expenses.monthly_report
1038 ;
1039

```

Result Grid

monthly_report	total_income	total_expenses
2024-03	24157.00	33159.00
2024-04	20676.00	36048.00
2024-05	26091.00	15396.50
2024-06	21171.00	18724.00
2024-07	37350.00	21761.58
2024-08	14045.00	10997.00
2024-12	4650.00	24216.18

Result 13

JOINING MONTHLY INCOME AND EXPENSES

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

3.2 Expense Breakdown by Category

- SQL query to categorize and summarize expenses.

MySQL Workbench

Navigator

SCHEMAS

personal_finance

Tables

debts

expenses

income

investments

savings

Views

monthly_financial_summ

Stored Procedures

GetFinancialHealth

Administration Schemas

Information

No object selected

SQL Editor

```

1032 sum(income_amount) as total_income
1033 from income
1034 group by monthly_report
1035 ;
1036
1037 # EXPENSE BREAKDOWN BY CATEGORY
1038 select
1039     expense_category, sum(expense_amount) as total_expenses
1040 from expenses
1041 group by expense_category
1042 ;
1043
1044 # INVESTMENT RETURNS (ROI CALCULATION)
1045 select
1046     investment_category, investment_amount, investment_return,
1047     FORMAT(((investment_return - investment_amount) / investment_amount) * 100, 2) as roi_percentage -- TO FORMAT RIGHT AWAY THE RESULT
1048 from investments
1049 ;
1050
1051

```

Result Grid

expense_category	total_expenses
Other	15259.58
Physical	84625.00
Rent	39368.50
Social	4305.00
Spiritual	5394.00
Travel	5528.00
Sum	19637.34

Result 14

EXPENSE BREAKDOWN BY CATEGORY

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

3.3 Investment Returns (ROI Calculation)

- Query to calculate return on investment (ROI) over time.

The screenshot displays the MySQL Workbench interface. The top toolbar includes options like File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. The left sidebar shows the 'SCHEMAS' panel with a tree view of databases and tables, including 'personal_finance' and 'monthly_financial_summ'. The main editor window shows a SQL query titled 'Return on Investment (ROI) Calculation'.

The SQL query is as follows:

```

group by expense_category
;

# INVESTMENT RETURNS (ROI CALCULATION)
select
investment_category, investment_amount, investment_return,
FORMAT(((investment_return - investment_amount) / investment_amount)*100,2) as roi_percentage -- TO FORMAT RIGHT AWAY THE RESULT
from investments

```

The query results are displayed in the 'Result Grid' panel, which shows a table with the following data:

investment_category	investment_amount	investment_return	roi_percentage
Pag-ibig Contribution	105.00	110.25	5.00
Pag-ibig Contribution	105.00	105.25	5.00
Banks	20.00	21.00	5.00
Banks	25.00	26.25	5.00
Banks	10.00	10.50	5.00
Lending	500.00	525.00	5.00
Lending	300.00	315.00	5.00
Lending	350.00	367.50	5.00
Pag-ibig Contribution	105.00	110.25	5.00
Pag-ibig Contribution	105.00	105.25	5.00
Banks	20.00	21.00	5.00
Banks	25.00	26.25	5.00

The bottom panel shows the 'Action Output' and 'Message' logs, indicating that the query was executed successfully and returned 15 rows.

3.4 Debt Summary with Interest Calculation

- Query to track debt amounts and interest payments.

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'personal_finance' database selected. The 'SQL Editor' pane in the center contains a SQL query:


```

  -- DEBT SUMMARY WITH INTEREST CALCULATION
  select
    debt_category, debt_amount, debt_return,
    (debt_return - debt_amount) as debt_interest
  from debts
  
```

 The query is highlighted with a red box. The 'Results' pane at the bottom shows the output of the query, which is a table with 4 columns: debt_category, debt_amount, debt_return, and debt_interest. The table contains 17 rows of data, including entries like 'Credit Card Debt', 'Debt from Family/Friends', and 'Personal Loans'. The results are also highlighted with a red box. The 'SQL Editor' pane has a toolbar at the top with various icons for editing and executing the query. The 'Results' pane has a toolbar with icons for viewing the results in different formats (Table, Grid, etc.). The 'Output' pane at the bottom shows the execution log, indicating that the query was executed successfully at 19:13:41 and 19:15:17. The 'SQL Editor' pane has a status bar at the bottom showing 'Read Only', 'Context Help', and 'Snippets'.

MySQL Workbench

unconnected x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

personal_finance

Tables

debts

expenses

investments

savings

Views

monthly_financial_summ

Stored Procedures

GetFinancialHealth

GetFinancialHealth2

Administration Schemas

Information

MySQL Portfolio

Limit to 500 rows

1050 from investments

1051 ;

1052

1053

1054

1055 -- DEBT SUMMARY WITH INTEREST CALCULATION

1056 select

1057 debt_category, debt_amount, debt_return,

1058 (debt_return - debt_amount) as debt_interest

1059 from debts

1060 ;

1061

1062

1063

1064

1065

1066

1067

1068

1069

1070

1071

1072

1073

1074

1075

1076

1077

1078

1079

1080

1081

1082

1083

1084

1085

1086

1087

1088

1089

1090

1091

1092

1093

1094

1095

1096

1097

1098

1099

1100

1101

1102

1103

1104

1105

1106

1107

1108

1109

1110

1111

1112

1113

1114

1115

1116

1117

1118

1119

1120

1121

1122

1123

1124

1125

1126

1127

1128

1129

1130

1131

1132

1133

1134

1135

1136

1137

1138

1139

1140

1141

1142

1143

1144

1145

1146

1147

1148

1149

1150

1151

1152

1153

1154

1155

1156

1157

1158

1159

1160

1161

1162

1163

1164

1165

1166

1167

1168

1169

1170

1171

1172

1173

1174

1175

1176

1177

1178

1179

1180

1181

1182

1183

1184

1185

1186

1187

1188

1189

1190

1191

1192

1193

1194

1195

1196

1197

1198

1199

1200

1201

1202

1203

1204

1205

1206

1207

1208

1209

1210

1211

1212

1213

1214

1215

1216

1217

1218

1219

1220

1221

1222

1223

1224

1225

1226

1227

1228

1229

1230

1231

1232

1233

1234

1235

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

1253

1254

1255

1256

1257

1258

1259

1260

1261

1262

1263

1264

1265

1266

1267

1268

1269

1270

1271

1272

1273

1274

1275

1276

1277

1278

1279

1280

1281

1282

1283

1284

1285

1286

1287

1288

1289

1290

1291

1292

1293

1294

1295

1296

1297

1298

1299

1300

1301

1302

1303

1304

1305

1306

1307

1308

1309

1310

1311

1312

1313

1314

1315

1316

1317

1318

1319

1320

1321

1322

1323

1324

1325

1326

1327

1328

1329

1330

1331

1332

1333

1334

1335

1336

1337

1338

1339

1340

1341

1342

1343

1344

1345

1346

1347

1348

1349

1350

1351

1352

1353

1354

1355

1356

1357

1358

1359

1360

1361

1362

1363

1364

1365

1366

1367

1368

1369

1370

1371

1372

1373

1374

1375

1376

1377

1378

1379

1380

1381

1382

1383

1384

1385

1386

1387

1388

1389

1390

1391

1392

1393

1394

1395

1396

1397

1398

1399

1400

1401

1402

1403

1404

1405

3.5 Sum for Each Table

- Aggregate calculations for each financial category.

The screenshot shows the MySQL Workbench interface. In the SQL editor, a query titled "# SUM FOR EACH TABLE" is written, which uses subqueries to calculate the sum of income, expenses, investments, savings, and debts. The query is highlighted with a red box. To the right of the query, the text "SUM for each table made" is written in red. Below the query, the "Result Grid" is displayed, showing a table with five columns: total_income, total_expense, total_investment, total_saving, and total_debt. The values are 148140.00, 349663.04, 4630.00, 22939.00, and 3796.00 respectively. The "Output" pane at the bottom shows the execution of the query, with a message indicating that 1 row(s) were returned.

```
# SUM FOR EACH TABLE
select
  (select SUM(income_amount) from income) as total_income,
  (select SUM(expense_amount) from expenses) as total_expense,
  (select SUM(investment_amount) from investments) as total_investment,
  (select SUM(saving_amount) from savings) as total_saving,
  (select SUM(debt_amount) from debts) as total_debt
;
```

total_income	total_expense	total_investment	total_saving	total_debt
148140.00	349663.04	4630.00	22939.00	3796.00

3.6 Total Net Worth Calculation

- Query to calculate overall financial standing.

The screenshot shows the MySQL Workbench interface. In the SQL editor, a query titled "# TOTAL NETWORTH CALCULATION" is written, which calculates the net worth by summing income, investments, and savings, and then subtracting expenses and debts. The query is highlighted with a red box. To the right of the query, the text "TOTAL NETWORTH Calculation" is written in red. Below the query, the "Result Grid" is displayed, showing a table with one column: net_worth. The value is -177750.04. The "Output" pane at the bottom shows the execution of the query, with a message indicating that 1 row(s) were returned.

```
# TOTAL NETWORTH CALCULATION
select
  (select SUM(income_amount) from income)
  - (select SUM(expense_amount) from expenses)
  + (select SUM(investment_amount) from investments)
  + (select SUM(saving_amount) from savings)
  - (select SUM(debt_amount) from debts) as net_worth
;
```

net_worth
-177750.04

📁 Step 4: Automate Reports with Views & Stored Procedures

4.1 Create a View for Monthly Financial Overview

- View to display key financial metrics.

The screenshot shows the MySQL Workbench interface with a SQL query editor. The query is creating a view named `monthly_financial_summary`. The query is as follows:

```
1081 ## AUTOMATE REPORTS WITH VIEWS & STORED PROCEDURES
1082 # Creating a VIEW for the Monthly Financial Overview
1083 CREATE VIEW monthly_financial_summary AS
1084 SELECT
1085     monthly_report, -- [IT WILL EXIST BECAUSE OF THE SUBQUERIES BELOW]
1086     SUM(total_income) AS total_income, -- [THIS ALSO WORKS BECAUSE IT SUMS UP THE VALUES FROM THE DIFFERENT UNION ALL RESULTS BELOW]
1087     SUM(total_expense) AS total_expense, -- [...THESE ARE CALLED OUTER QUERIES]
1088     SUM(total_investment) AS total_investment, -- [...THEY ARE ALIASES DEFINED INSIDE EACH SELECT STATEMENT OF THE UNION ALL QUERY]
1089     SUM(total_saving) AS total_saving,
1090     SUM(total_debt) AS total_debt
1091 FROM (
1092     -- Income
1093     SELECT DATE_FORMAT(income_date, '%Y-%m') AS monthly_report,
1094            SUM(income_amount) AS total_income,
1095            0 AS total_expense,
1096            0 AS total_investment,
1097            0 AS total_saving,
1098            0 AS total_debt
1099     FROM income
1100     GROUP BY monthly_report
1101
1102     UNION ALL
1103
1104     -- Expenses
1105     SELECT DATE_FORMAT(expense_date, '%Y-%m') AS monthly_report,
1106            0 AS total_income,
1107            SUM(expense_amount) AS total_expense,
1108            0 AS total_investment,
1109            0 AS total_saving,
```

The query is highlighted with a red box. A red text overlay on the right side of the query editor reads: "CREATING A VIEW for the Monthly Financial Overview".

The screenshot shows the MySQL Workbench interface with the same SQL query editor. The query is now complete, including the `UNION ALL` and the final `select * from monthly_financial_summary;` statement. The query is highlighted with a red box. Below the query editor, the "Result Grid" is displayed, showing the data for the `monthly_financial_summary` view. The data is as follows:

monthly_report	total_income	total_expense	total_investment	total_saving	total_debt
2024-03	24157.00	57316.00	0.00	2545.00	804.00
2024-04	20676.00	36724.00	0.00	779.00	150.00
2024-05	26091.00	41487.68	0.00	70.00	0.00
2024-06	21171.00	39995.08	0.00	93.00	0.00
2024-07	37350.00	59110.74	110.00	679.00	0.00
2024-08	14045.00	25042.20	0.00	71.00	0.00
2024-12	4650.00	28866.09	2300.00	1157.00	0.00
2024-01	0.00	13258.00	2220.00	208.00	2384.00
2024-02	0.00	12138.00	0.00	398.00	458.00
2024-09	0.00	18313.25	0.00	14254.00	0.00

4.2 Stored Procedure for Financial Health by Month

- Automated query to analyze finances month by month.

The screenshot shows the MySQL Workbench interface with the 'Creation of STORED PROCEDURE for Financial Health by Month' title. The SQL editor contains the following code:

```
# STORED PROCEDURE FOR FINANCIAL HEALTH BY ONE MONTH
DELIMITER //
-- Used to change the default delimiter ";" to "/"
CREATE PROCEDURE GetFinancialHealth(IN month_year VARCHAR(7))
BEGIN
    SELECT
        (SELECT SUM(income_amount) FROM income WHERE DATE_FORMAT(income_date, 'SY-%m') = month_year) AS total_income,
        (SELECT SUM(expense_amount) FROM expenses WHERE DATE_FORMAT(expense_date, 'SY-%m') = month_year) AS total_expenses,
        (SELECT SUM(investment_amount) FROM investments WHERE DATE_FORMAT(investment_date, 'SY-%m') = month_year) AS total_investments,
        (SELECT SUM(saving_amount) FROM savings WHERE DATE_FORMAT(saving_date, 'SY-%m') = month_year) AS total_savings,
        (SELECT SUM(debt_amount) FROM debts WHERE DATE_FORMAT(debt_date, 'SY-%m') = month_year) AS total_debts;
END //
DELIMITER ;
-- changing the delimiter back to its default ";" again

CALL GetFinancialHealth('2024-01'); -- Checking if the code works
CALL GetFinancialHealth('2024-03');
```

The output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
28	19:22:25	CREATE VIEW monthly_financial_summary AS SELECT monthly_report. -- IT WILL EXIST BECAUSE O...	Error Code: 1050. Table 'monthly_financial_summary' already exists	0.015 sec
29	19:26:49	select * from monthly_financial_summary LIMIT 0.500	13 row(s) returned	0.000 sec / 0.000 sec
30	19:27:03	select * from monthly_financial_summary LIMIT 0.500	13 row(s) returned	0.000 sec / 0.000 sec
31	19:28:00	CREATE PROCEDURE GetFinancialHealth(IN month_year VARCHAR(7)) BEGIN SELECT (SELEC...	Error Code: 1304. PROCEDURE GetFinancialHealth already exists	0.000 sec
32	19:29:44	CALL GetFinancialHealth('2024-03')	1 row(s) returned	0.016 sec / 0.000 sec
33	19:29:48	CALL GetFinancialHealth('2024-01')	1 row(s) returned	0.000 sec / 0.000 sec
34	19:30:13	drop procedure if exists GetFinancialHealth	0 row(s) affected	0.000 sec
35	19:30:20	CREATE PROCEDURE GetFinancialHealth(IN month_year VARCHAR(7)) BEGIN SELECT (SELEC...	0 row(s) affected	0.000 sec

4.3 Stored Procedure for Financial Health Over Multiple Months

- Extended analysis over a chosen period.

The screenshot shows the MySQL Workbench interface with the same SQL code as in the previous screenshot. The output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
28	19:22:25	CREATE VIEW monthly_financial_summary AS SELECT monthly_report. -- IT WILL EXIST BECAUSE O...	Error Code: 1050. Table 'monthly_financial_summary' already exists	0.015 sec
29	19:26:49	select * from monthly_financial_summary LIMIT 0.500	13 row(s) returned	0.000 sec / 0.000 sec
30	19:27:03	select * from monthly_financial_summary LIMIT 0.500	13 row(s) returned	0.000 sec / 0.000 sec
31	19:28:00	CREATE PROCEDURE GetFinancialHealth(IN month_year VARCHAR(7)) BEGIN SELECT (SELEC...	Error Code: 1304. PROCEDURE GetFinancialHealth already exists	0.000 sec
32	19:29:44	CALL GetFinancialHealth('2024-03')	1 row(s) returned	0.016 sec / 0.000 sec
33	19:29:48	CALL GetFinancialHealth('2024-01')	1 row(s) returned	0.000 sec / 0.000 sec
34	19:30:13	drop procedure if exists GetFinancialHealth	0 row(s) affected	0.000 sec
35	19:30:20	CREATE PROCEDURE GetFinancialHealth(IN month_year VARCHAR(7)) BEGIN SELECT (SELEC...	0 row(s) affected	0.000 sec

The result grid shows the following data:

	total_income	total_expenses	total_investments	total_savings	total_debts
1	24157.00	57316.00	2545.00	804.00	

Now, a working stored procedure.

4.4 Stored procedure for financial health over multiple months

The screenshot shows the MySQL Workbench interface. In the left sidebar, the 'Schemas' panel is open, and 'GetFinancialHealth2' is highlighted under the 'Stored Procedures' folder. The main editor displays the SQL code for creating the stored procedure. The code is as follows:

```
-- STORED PROCEDURE FOR FINANCIAL HEALTH OVER MULTIPLE MONTHS
DELIMITER //
CREATE PROCEDURE GetFinancialHealth2(IN start_month VARCHAR(7), IN end_month VARCHAR(7))
BEGIN
    SELECT
        month_year,
        SUM(total_income) AS total_income,
        SUM(total_expenses) AS total_expenses,
        SUM(total_investments) AS total_investments,
        SUM(total_savings) AS total_savings,
        SUM(total_debts) AS total_debts
    FROM (
        SELECT
            DATE_FORMAT(income_date, '%Y-%m') AS month_year,
            SUM(income_amount) AS total_income,
            0 AS total_expenses,
            0 AS total_investments,
            0 AS total_savings,
            0 AS total_debts
        FROM income
        WHERE DATE_FORMAT(income_date, '%Y-%m') BETWEEN start_month AND end_month
    ) AS financial_data
    GROUP BY month_year
    ORDER BY month_year;
END //
DELIMITER ;
```

The output window at the bottom shows the execution results. The final line indicates that the stored procedure was created successfully, affecting 0 rows.

#	Time	Action	Message	Duration / Fetch
36	19:33:41	CALL GetFinancialHealth(2024-03)	1 row(s) returned	0.000 sec / 0.000 sec
37	19:33:46	CALL GetFinancialHealth(2024-01)	1 row(s) returned	0.000 sec / 0.000 sec
38	19:33:48	CALL GetFinancialHealth(2024-03)	1 row(s) returned	0.000 sec / 0.000 sec
39	19:33:58	CALL GetFinancialHealth(2024-03)	1 row(s) returned	0.000 sec / 0.000 sec
40	19:37:04	drop procedure if exists GetFinancialHealth2	0 row(s) affected	0.000 sec
41	19:37:17	CREATE PROCEDURE GetFinancialHealth2(IN start_month VARCHAR(7), IN end_month VARCHAR(7)) BE...	0 row(s) affected	0.000 sec

The screenshot shows the MySQL Workbench interface with the stored procedure being executed. The SQL editor shows the following code:

```
SELECT
    DATE_FORMAT(debt_date, '%Y-%m') AS month_year,
    0 AS total_income,
    0 AS total_expenses,
    0 AS total_investments,
    0 AS total_savings,
    SUM(debt_amount) AS total_debts
FROM debts
WHERE DATE_FORMAT(debt_date, '%Y-%m') BETWEEN start_month AND end_month
GROUP BY month_year
) AS financial_data
GROUP BY month_year
ORDER BY month_year;
END //
DELIMITER ;
```

The output window shows the execution results. The final line indicates that the stored procedure was executed successfully, affecting 0 rows.

month_year	total_income	total_expenses	total_investments	total_savings	total_debts
2024-01	0.00	13258.00	2220.00	208.00	2384.00

The text "CHECKING..." is displayed in green next to the result grid.

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- ▼ savings
- ▼ Views
 - monthly_financial_summ
- ▼ Stored Procedures
 - GetFinancialHealth
 - GetFinancialHealth2
- ▼ sakila
 - Tables
 - Views
 - Stored Procedures

Administration Schemas

Information

No object selected

MySQL Portfolio

Limit to 500 rows

```
1240 SELECT
1241     DATE_FORMAT(debt_date, '%Y-%m') AS month_year,
1242     0 AS total_income,
1243     0 AS total_expenses,
1244     0 AS total_investments,
1245     0 AS total_savings,
1246     SUM(debt_amount) AS total_debts
1247 FROM debts
1248 WHERE DATE_FORMAT(debt_date, '%Y-%m') BETWEEN start_month AND end_month
1249 GROUP BY month_year
1250 ) AS financial_data
1251 GROUP BY month_year
1252 ORDER BY month_year;
1253 END //
1254 DELIMITER ;
1255
1256 CALL GetFinancialHealth2('2024-01', '2024-01');
1257 CALL GetFinancialHealth2('2024-01', '2024-03');
1258
```

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

month_year	total_income	total_expenses	total_investments	total_savings	total_debts
2024-01	0.00	13258.00	2220.00	208.00	2384.00
2024-02	0.00	12138.00	0.00	398.00	458.00
2024-03	24157.00	57316.00	0.00	2545.00	804.00

Result 31

Output

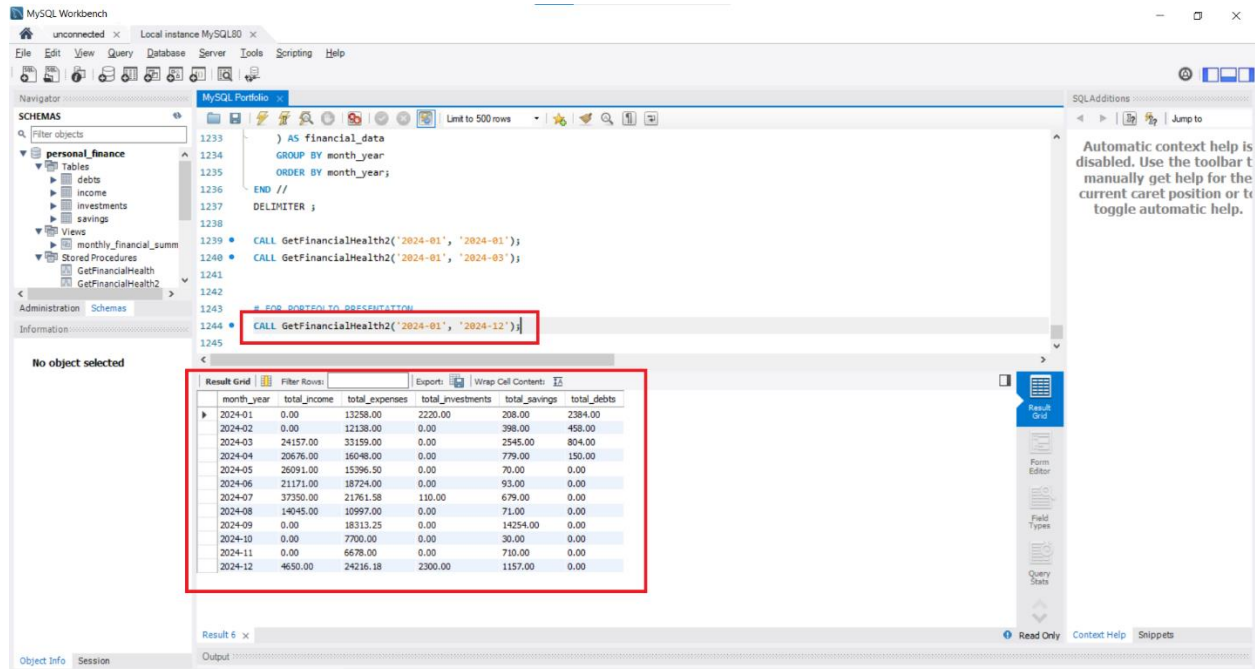
Read Only Context Help Snippets

Now, another fully functional stored procedure for financial health over multiple months.

DATA INSIGHTS

1. What is the general financial overview for each month in 2024?

- The monthly financial overview for 2024 highlights trends in income, expenses, investments, savings, and debts. It provides a clear snapshot of financial performance, showing growth, stability, or areas needing adjustment. This helps track progress and make informed financial decisions throughout the year.



MySQL Workbench interface showing a SQL query and its results. The query is a stored procedure call for financial health. The results table shows monthly financial data for 2024.

month_year	total_income	total_expenses	total_investments	total_savings	total_debts
2024-01	0.00	13258.00	2220.00	208.00	2384.00
2024-02	0.00	12138.00	0.00	398.00	458.00
2024-03	24157.00	33159.00	0.00	2545.00	804.00
2024-04	20676.00	36048.00	0.00	779.00	150.00
2024-05	26991.00	15396.50	0.00	70.00	0.00
2024-06	21171.00	18724.00	0.00	93.00	0.00
2024-07	37350.00	21761.58	110.00	679.00	0.00
2024-08	14045.00	10997.00	0.00	71.00	0.00
2024-09	0.00	18313.25	0.00	14254.00	0.00
2024-10	0.00	7700.00	0.00	30.00	0.00
2024-11	0.00	6678.00	0.00	710.00	0.00
2024-12	4650.00	24216.18	2300.00	1157.00	0.00

2. How are expenses distributed across different categories, and what are the 3 major spending areas?

- Expenses are categorized into distinct groups (e.g., physical, travel, social, etc.), and as shown in the results, the top three spending categories are **'Physical'**, which ranks highest with a total of ₱84,625 spent, followed by **'Rent'** at ₱39,368, and **'Family'** with ₱28,416.18, with the remaining categories following in order.
 - By breaking down spending in these categories, we can identify which areas consume the most resources and explore opportunities to reduce costs, providing valuable insights for optimizing budgeting strategies.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

1239 CALL GetFinancialHealth2('2024-01', '2024-01');
1240 CALL GetFinancialHealth2('2024-01', '2024-03');
1241
1242
1243 # FOR PORTFOLIO PRESENTATION
1244 CALL GetFinancialHealth2('2024-01', '2024-12');
1245
1246 select expense_category, SUM(expense_amount) as total_expenses
1247 from expenses
1248 group by expense_category
1249 order by total_expenses desc
1250
1251

```

The query results are displayed in the Result Grid:

expense_category	total_expenses
Physical	84625.00
Rent	39368.50
Family	28416.18
Other	18627.25
Travel	5528.00
Spiritual	5394.00
Social	4305.00

3. How much have investments earned in terms of returns?

- The ROI formula is used to calculate investment returns by comparing the total amount invested to the returns generated. According to the query, the total ROI for 2024 is **P231.50**, which accounts for **5% of the total investment**.
 - This calculation helps assess the performance of various investments over the year, providing valuable insights for making strategic decisions on where to allocate future funds.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

1274 FORMAT(((investment_return - investment_amount) / investment_amount)*100,2) as roi_percentage -- TO FORMAT RIGHT AWAY THE RESULT
1275 from investments
1276
1277
1278 select
1279     SUM(investment_return) as total_return,
1280     SUM(investment_amount) as total_investment,
1281     SUM(investment_return) - SUM(investment_amount) as ROI,
1282     FORMAT(((SUM(investment_return) - SUM(investment_amount)) / SUM(investment_amount)) * 100,2) as ROI_percentage
1283 from investments
1284
1285

```

The query results are displayed in the Result Grid:

total_return	total_investment	ROI	ROI_percentage
4861.50	4630.00	231.50	5.00

4. What is the total debt and interest paid?

- The debt summary query calculates the total outstanding debt, while interest calculations provide insights into the cost of borrowing over time. This helps in managing repayment strategies effectively. For 2024, the total debt stands at ₦3,796, with ₦189.90 paid in interest, representing 5% of the total debt.*

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
select
  SUM(debt_return) as total_debt_return,
  SUM(debt_amount) as total_debt_amount,
  SUM(debt_return) - SUM(debt_amount) as interest,
  FORMAT(((SUM(debt_return) - SUM(debt_amount)) / SUM(debt_amount)) * 100,2) as interest_percentage
from debts
```

The Results window displays the following data:

total_debt_return	total_debt_amount	interest	interest_percentage
3985.80	3796.00	189.80	5.00

5. What is the total net worth?

- Total net worth is calculated by adding up assets (e.g., savings, investments, etc.) and subtracting liabilities (e.g., expenses, debts). As shown in the query, the total net worth for the year 2024 is ₦-29,610.51, indicating poor financial management during this period.*
 - By tracking net worth over time, we can evaluate overall financial progress and assess the impact of income, spending, and investment decisions.*

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
select
  (select SUM(income_amount) from income)
  - (select SUM(expense_amount) from expenses)
  + (select SUM(investment_amount) from investments)
  + (select SUM(saving_amount) from savings)
  - (select SUM(debt_amount) from debts) as net_worth
```

The Results window displays the following data:

net_worth
-29610.51

KEY INSIGHTS

- Monthly financial tracking in 2024 revealed varying trends in income, spending, savings, and debt, providing a clearer picture of overall financial flow and stability.
- The top 3 spending categories were **Physical** (₱84,625), **Rent** (₱39,368), and **Family** (₱28,416.18), indicating where most resources were allocated.
- Total investment returns reached **₱231.50**, representing a modest **5% ROI**, which helps evaluate the effectiveness of investment decisions.
- The total debt stood at **₱3,796**, with **₱189.90** in interest paid — highlighting the cost of borrowing and areas for improving repayment strategies.
- The **net worth for 2024 was negative ₱29,610.51**, pointing to overspending or underperformance in savings and asset growth throughout the year.

RECOMMENDATION

- To improve overall financial health, consider reviewing and adjusting high-expense categories like Physical and Rent, while increasing focus on consistent savings and better-performing investments. Monitoring financial data monthly and reallocating funds strategically could help shift net worth into a positive trajectory.

REFLECTION & NEXT STEPS

- This project strengthened my ability to write complex SQL queries and structure a database from scratch, helping me turn raw personal finance data into meaningful insights. It gave me hands-on experience applying SQL to real-world questions. Moving forward, I'd like to automate the data entry process using tools like Google Sheets and integrate a dashboard for live tracking and more dynamic analysis.